GUI in Python

In Python, there are several libraries available for creating Graphical User Interfaces (GUIs) to build interactive applications. Tkinter is one such library that allows to build GUI.

Tkinter:

Tkinter is a standard Python library for creating GUI applications. It provides a set of widgets, such as buttons, labels, text boxes, and frames, which can be arranged to create a user interface. Tkinter is easy to use and has good documentation.

The root Window

In GUI programming, the root window is the main window or the top-level window in an application's window hierarchy. It serves as the parent or container for all other GUI elements, such as buttons, labels, menus, and other widgets.

Creating root window

Root window is created by creating an object of Tk class.

```
from tkinter import *
root = Tk()
root.mainloop()
```

Steps involved in creating a GUI application

To create a GUI application using Tkinter, the following steps are typically involved:

1. Import the Tkinter module: Start by importing the Tkinter module in your Python script to access the necessary classes and functions for GUI programming.
2. Create the root window: Create an instance of the Tk() class to create the main window or root window of your application.
3. Create a Canvas or Frame: Canvas and frames are child windows in root window. Canvas is used to draw shapes and Frame is used to place the widgets.
4. Add widgets: Use various Tkinter classes (such as Label, Button, Entry, etc.) to create and add widgets to the root window. These widgets represent the different GUI elements of your application. 5. Define event handlers: Specify the functions that should be executed when specific events occur, such as button clicks or key presses.
6. Start the event loop: Finally, call the mainloop() method on the root window to start the main event loop. This loop listens for events, handles user interactions, and keeps the GUI application running until the window is closed.

Example:

```
from tkinter import *
root = Tk()
f = Frame(root, height=400, width=300)
def button_click():
    print("Button clicked!")   # properly indented
button = Button(f, text="Click Me!", command=button_click)
button.pack()
f.pack()
root.mainloop()
```

Working with Containers

A container is a component that is used as a place where drawings and widgets can be displayed. The two main containers are: Canvas: Canvas is a container is used to draw shapes like lines, arcs and circles Frame: Frame is a container is used to display widgets like buttons, Labels, Entry etc.

Canvas

A canvas is used to create GUI to allow us to create and manipulate graphical objects, such as shapes, lines, images, and text.

```
from tkinter import *
root = Tk()
c = Canvas(root, width=400, height=300)
line_id = c.create_line(100, 100, 200, 200)
c.pack()
root.mainloop()
```

Frame
Frame is a container is used to display widgets like buttons, Labels, Entry etc. Creating a frame:

```
from tkinter import *
root = Tk()
f = Frame(root, height=400, width=300)
f.pack()
root.mainloop()
```

1. Pack Layout Manager:
The pack() method is a simple layout manager that arranges widgets in a horizontal or vertical sequence. It automatically adjusts the size of widgets based on their content and places them one after another.

```
from tkinter import *
root = Tk()
f = Frame(root, height=300, width=300)
f.propagate(0)   # prevents resizing
f.pack()
b1 = Button(f, text='Button 1', bg='blue')
b2 = Button(f, text='Button 2', bg='red')
b1.pack(fill=X, padx=10, pady=10)
b2.pack(fill=X)
root.mainloop()
```

Right and Top

```
from tkinter import *
root = Tk()
f = Frame(root, height=300, width=300)
f.propagate(0)
f.pack()
b1 = Button(f, text='Button 1')
b2 = Button(f, text='Button 2')
b1.pack(side=RIGHT)
b2.pack(side=TOP)
root.mainloop()
```

2. Grid Layout Manager:
The grid() method allows you to create a grid-like structure to position widgets in rows and columns.

```
from tkinter import *
root = Tk()
f = Frame(root, height=400, width=300)
f.pack()
label1 = Label(f, text='Label 1')
label2 = Label(f, text='Label 2')
label3 = Label(f, text='Label 3')
label1.grid(row=0, column=0)
label2.grid(row=0, column=1)
label3.grid(row=1, column=0, columnspan=2)
root.mainloop()
```

3. Place Layout Manager:
The place() method allows you to specify the exact coordinates (x, y) of each widget within the window.

```
from tkinter import *
root = Tk()
f = Frame(root, height=400, width=300)
f.pack()
```

```
label1 = Label(f, text='Label 1')
label2 = Label(f, text='Label 2')
label3 = Label(f, text='Label 3')
label1.place(x=10, y=10)
label2.place(x=50, y=50)
label3.place(x=100, y=100)
root.mainloop()
```

Widgets

1)Text

Creating a Text widget:

```
t=Text(root,width=20,height=10,font=('verdana',14), fg='vlue')
```

Example

```
from tkinter import *
root = Tk()
t = Text(root, width=20, height=10, font=('verdana', 14), fg='blue')
t.pack()
root.mainloop()
```

Inserting text into Text widget using insert() method:

Example

```
from tkinter import *
root = Tk()
t = Text(root, width=30, height=10)
t.pack()
# Insert text at beginning
t.insert("1.0", "Hello World\n")
# Insert text at end
t.insert(END, "Welcome to Tkinter")
root.mainloop()
```

Marking some part of the text as a tag:

```
from tkinter import *
root = Tk()
t = Text(root, width=40, height=10)
t.pack()
t.insert(END, "Hello World\nWelcome to Tkinter")
# Apply tag to part of second line
t.tag_add('start', '2.0', '2.7')
t.tag_config('start', background='red', foreground='white',
        font=('Verdana', 20, 'bold italic'))
root.mainloop()
```

Display an image using image_create() method

```
from tkinter import *
root = Tk()
t = Text(root, width=40, height=10)
t.pack()
# Load image (only PNG/GIF supported directly)
img = PhotoImage(file="F:\SBC\SEP\BCA SEP\II BCA\IV SEM\PYTHON\LAB PGMS\imgi.gif")
# Insert image into Text widget
t.image_create(END, image=img)
root.mainloop()
```

Adding a scrollbar to the Text widget:
We can add vertical or horizontal scrollbar. First create a scrollbar object and then configure it to the Text widget. Finally attach scroll bar to the root window.

```
from tkinter import *
root = Tk()
t = Text(root, width=40, height=10)
t.pack(side=LEFT)
s = Scrollbar(root, orient=VERTICAL, command=t.yview)
t.configure(yscrollcommand=s.set)
s.pack(side=RIGHT, fill=Y)
# Add lots of text to test scrolling
for i in range(50):
    t.insert(END, f"Line {i}\n")
root.mainloop()
```

Label
Label represents text that is displayed in the frame or container. Display text that cannot be modified.

```
from tkinter import *
root = Tk()
label = Label(root, text="Hello, World!")
label.pack()
root.mainloop()
```

Button
A push button is a component that performs some action when clicked. Creating a button: b=Button(f,text='My Button', width=15, heigh=3, fg='blue')

```
from tkinter import *
root = Tk()
f = Frame(root, height=300, width=300)
f.propagate(0)
f.pack()
def buttonClick(event):
    print('Button is clicked')
b = Button(f, text='My Button', width=15, height=3, fg='blue')
b.bind('<Button-1>', buttonClick)
b.pack()
root.mainloop()
```

b. Using command option:
Command option in the Button constructor link the push button with event handler function.

```
from tkinter import *
root = Tk()
f = Frame(root, height=300, width=300)
f.propagate(0)
f.pack()
def buttonClick():
    print('Button is clicked')
b = Button(f, text='Submit', width=15, height=3, fg='blue', command=buttonClick)
b.pack()
root.mainloop()
```

Entry
The Entry widget in Tkinter is used to create a single-line input field where users can enter text or numeric values. Creating an Entry widget: e1 = Entry(f, width=25,fg=’blue’, show=’*’)

```
from tkinter import *
root = Tk()
f = Frame(root, height=300, width=300)
f.propagate(0)
f.pack()
e1 = Entry(f, width=25, fg='blue')
e1.place(x=50, y=100)
def display(event):
    text = e1.get()
    print(text)
# Bind Enter key to function
e1.bind("<Return>", display)
root.mainloop()
```

Listbox

List books is useful to display a list of items in a box so that the user can select one or more items.
Creating a Listbox: box1=Listbox(f,height=8,width=20,activestyle=UNDERLINE,selectmode=SINGLE)

Inseting items into the list using insert() method:
box1.insert(0,'Mysore')

Example

```
from tkinter import *
root = Tk()
# Create Listbox
lb = Listbox(root)
# Add items
lb.insert(END, "Mysore")
lb.insert(END, "Bangalore")
lb.insert(END, "Delhi")
# Show Listbox
lb.pack()
root.mainloop()
```

Example 2

```
from tkinter import *
root = Tk()
f = Frame(root, height=300, width=300)
f.propagate(0)
f.pack()
box1 = Listbox(f, height=8, width=20,
               activestyle=UNDERLINE,
               selectmode=SINGLE)
box1.insert(0, 'Mysore')
box1.insert(1, 'Bangalore')
box1.insert(2, 'Delhi')
box1.pack()
def on_select(event):
    lst = []
    indexes = box1.curselection()
    for i in indexes:
        lst.append(box1.get(i))
    print(lst)
# Bind event (outside function)
box1.bind('<<ListboxSelect>>', on_select)
```

root.mainloop()

Checkbutton
Checkbutton is used when user want to select one or more options from available group of options. Checkbuttons are displayed in the form of square shaped boxes.

Creating a Checkbutton:
c1=Checkbutton(f, text='Java',variable=var1,command=display)

Example
```
from tkinter import *
root = Tk()
f = Frame(root, height=300, width=300)
f.propagate(0)
f.pack()
var1 = IntVar()
def display():
    if var1.get():
        print('Checkbutton is checked')
    else:
        print('Checkbutton is unchecked')

c1 = Checkbutton(f, text='Java', variable=var1, command=display)
c1.pack()
root.mainloop()
```

Radiobutton
Radiobutton is used to select only one option from a group of options. A radiobutton is displayed as round shaped button.

Creating a radiobutton:
r1=Radiobutton(f, text='Male',variable=var1,command=display,value=1)

Example
```
from tkinter import *
root = Tk()
f = Frame(root, height=300, width=300)
f.propagate(0)
f.pack()
var1 = IntVar()
def display():
    print('The value of the radio button is', var1.get())
r1 = Radiobutton(f, text='Male', variable=var1, command=display, value=1)
r2 = Radiobutton(f, text='Female', variable=var1, command=display, value=2)
r1.pack()
r2.pack()
root.mainloop()
```

Scrollbar
A scroll bar is used to scroll the text in other widget such as Text,Canvas, Frame or Listbox. Types of scroll bars are: horizontal and vertical.
The horizontal scroll bar is used to view the text from left to right.
The vertical scroll bar is used to scroll the text from top to bottom.

Example
```
from tkinter import *
root = Tk()
```

```
text = Text(root, height=10, width=30)
text.pack(side=LEFT)
scroll = Scrollbar(root)
scroll.pack(side=RIGHT, fill=Y)
# Connect scrollbar and text
scroll.config(command=text.yview)
text.config(yscrollcommand=scroll.set)
# Add sample text
for i in range(20):
    text.insert(END, f"Line {i}\n")
root.mainloop()
```

Spinbox

A Spinbox widget allows the user to select values from a given set of values. The values may be range of numbers or a fixed set of string.

Creating a Spinbox:

s1 = Spinbox(f, from_=1, to=10)

Example

```
from tkinter import *
root = Tk()
f = Frame(root, height=300, width=300)
f.propagate(0)
f.pack()
s1 = Spinbox(f, from_=1, to=10)
s1.pack()
def show():
    value = s1.get()
    print('Selected value:', value)
Button(f, text="Show Value", command=show).pack()
root.mainloop()
```